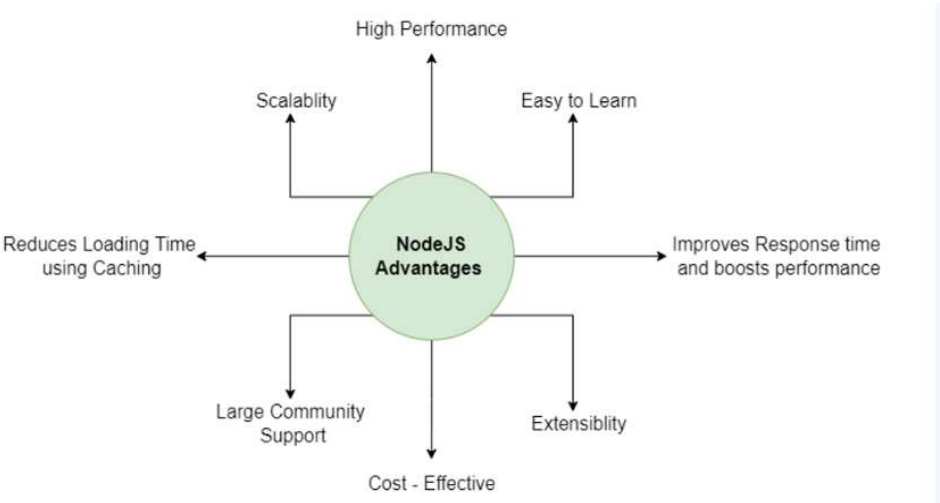
What is Node.js?

- Node.js is an open-source server side runtime environment built on Chrome's V8 JavaScript engine.
- It provides an **event driven, non-blocking (asynchronous) I/O and cross-platform runtime environment** for building highly scalable server-side application using JavaScript.
- **Event-Driven**: Node.js listens for events and processes them using callback functions, which makes it highly responsive and efficient.
- **Non-Blocking (Asynchronous) I/O**: Node.js performs I/O operations asynchronously, allowing other tasks to proceed without waiting for the I/O operation to complete, leading to better performance.
- **Cross-Platform Runtime Environment**: Node.js runs on various operating systems, enabling code portability and simplifying development and deployment.
- Node.js can be used to build **different types of applications such as command line application, web application, real-time chat application, REST API server** etc.
- However, it is mainly used to **build network programs** like web servers, similar to PHP, Java, or ASP.NET.
- Moreover, it operates on a **single threaded event based loop** to make all executions non-blocking.
- However, you cannot consider Node.js as not a framework and it has nothing to do with the PHP, .NET or JAVA.
- Key Features:
    - **Event-Driven Architecture**: Asynchronous and non-blocking I/O.
    - **Single-Threaded Model**: Uses a single thread for handling multiple connections.
    - **NPM (Node Package Manager)**: Rich ecosystem of libraries and modules.

Node.js = Runtime Environment + JavaScript Library

Advantages of NodeJS



High Performance:

- One of the most important features of node.js is the ability to **create lightning-fast apps that produce results in seconds.**
- Its event-driven, single-threaded design rapidly processes **several simultaneous requests without blocking the RAM**. Additionally, its event-loop and **non-blocking I/O operations** allow code to be executed at a rate that indirectly influences the application's overall performance.

Scalability

- One of the Node.js advantages is that developers can **easily grow applications in both horizontal and vertical orientations.** The applications can be horizontally scaled by **adding additional nodes** to the existing system.
- Because Node.js is designed to handle many connections concurrently, it is well-suited for horizontal scaling. You can add more servers to distribute the workload without significant changes to the codebase - If your application is receiving more traffic, you can deploy additional instances of your application on multiple servers. A load balancer can then distribute the incoming requests across these instances, ensuring that no single server is overwhelmed.
- Furthermore, during the vertical scaling of the application, **Node.js allows you to add extra resources to single nodes.** As a result, it is extremely scalable and offers a superior alternative to existing JavaScript servers - Node.js is efficient in terms of CPU and memory usage, which allows you to effectively utilize more powerful hardware to handle more operations within a single machine.

Easy to Learn

- Most frontend developers **are familiar with JavaScript** because it is one of the most extensively used programming languages.
- It will be lot easier for them to get started using Node.js on the backend. It is simpler to understand Node.js, and working with it takes less time.

Cost-Effective

- Node.js allows programmers to develop **server-side JavaScript and frontend JavaScript codes with simplicity.** One of the major node.js advantages is that it eliminates the need for two resource teams, saving time, money, and energy for overall project development.
- LinkedIn's development team switched from Ruby on Rails to Node.js, and the benefits of Node.js helped them gain performance. They lowered the number of servers from 30 to only three. They began delegating and utilizing resources more effectively, changing their attention from debugging to app development.

Extensibility

- Node.js offers a lot of extensibility, which means it can be changed and improved to fit unique requirements.
- Data may be sent between the **web server and the client using the JSON format**. It also has APIs for constructing HTTP, TCP, and DNS servers, among other features.

Large Community Support

- Millions of developers actively contribute to the Node.js community. To address even the most bizarre development difficulties, anticipate widespread cooperation from development specialists all across the world.
- NPM is the world's most comprehensive package manager registry. It includes a number of tools and libraries that you may use right away in your project. Developers will also have access to a wide pool of resources on **GitHub, including ready–to–use solutions, scripts, modules, libraries, and registries created by developers to help them do more with less**.
- Many big tech companies, including Amazon, Google, Facebook, and Netflix, contribute significantly to the node.js community. They've made significant contributions in the shape of several open-source projects. Acceptance by tech giants and the development community ensures the presence and expansion of technology in the foreseeable future.

Improves Response time and boosts performance

- A constant stream of queries impacts an app's response time and performance. If your IT stack isn't ready to manage a high volume of requests, the value of your product will rapidly decline.
- The **single-threaded event-loop approach in Node.js provides a non-blocking asynchronous design.** This design uses fewer resources since it does not spawn more threads. It improves the responsiveness of your application by dealing with several concurrent users.
- When Netflix used Java and JavaScript, the startup time was 40 minutes. They then moved to node.js and succeeded in cutting the startup time to under 40 seconds.

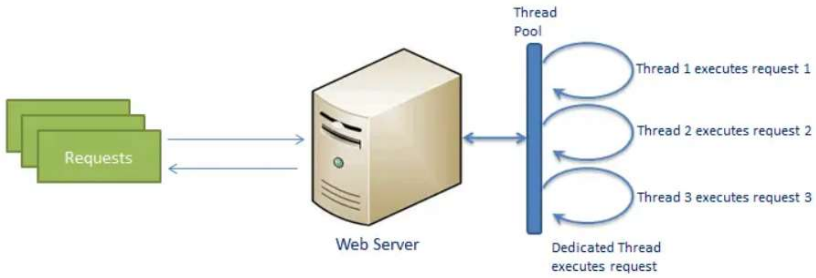
Reduces Loading Time using caching

- The **Node.js Caching module makes it simple for developers to decrease task workload** and code re-execution. It is one of the major node.js advantages.
- As a result, the initial module of the web application is cached in the in-app memory each time it is accessed. A user may browse online sites quickly without having to wait long.
- Helps in building the cross-functional teams
- You can use Node.js to construct cross-platform real-time web apps using platforms like Electron and NW.js. This eliminates the need to write separate code for different desktop versions such as Windows, Linux, and macOS.

This powerful technique is used in web development using Javascript frameworks like AnjularJS, Vue.js, and React for the frontend and Node.js for the backend. It has enabled full-stack developers to take advantage of the benefits and opportunities that node.js provides.

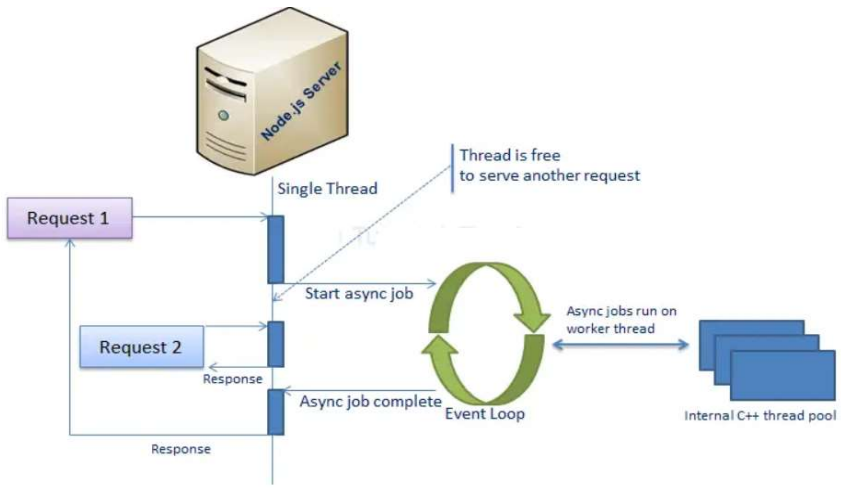
Node.js Process Model vs Traditional web server model

1. Traditional Web Server Model
   - In the traditional web server model, each request is handled by a dedicated thread from the thread pool.
   - If no thread is available in the thread pool at any point of time, then the request waits till the next available thread.
   - Dedicated thread executes a particular request and does not return to thread pool until it completes the execution and returns a response.



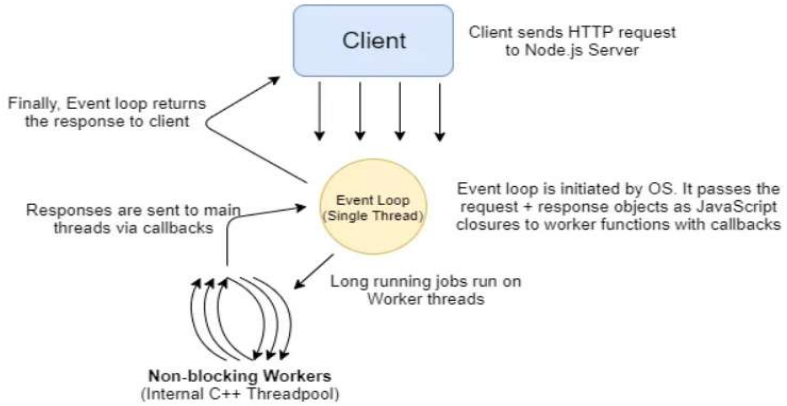
Node.js Process Model

- Node.js processes user requests differently when compared to a traditional web server model. Node.js runs in a single process and the application code runs in a single thread and thereby needs less resources than other platforms.
- All the user requests to your web application will be handled by a single thread and all the I/O work or long running job is performed asynchronously for a particular request. So, this **single thread** doesn't have to wait for the request to complete and is free to handle the next request. When asynchronous I/O work completes then it processes the request further and sends the response.
- An **event loop** is constantly watching for the events to be raised for an asynchronous job and executing callback function when the job completes.
- Internally, Node.js uses libev for the event loop which in turn uses **internal C++ thread pool** to provide asynchronous I/O.



Node.js Process Model

- Node.js process model increases the performance and scalability with a few requirements. Node.js is not fit for an application which performs CPU-intensive operations like image processing or other heavy computation work because it takes time to process a request and thereby blocks the single thread.



Installation Node.js

Node.js development environment can be setup in Windows, Mac, Linux and Solaris. The following tools/SDK are required for developing a Node.js application on any platform.

1. Node.js
2. Node Package Manager (NPM)
3. IDE (Integrated Development Environment) or TextEditor

NPM (Node Package Manager) is included in Node.js installation since Node version 0.6.0., so there is no need to install it separately.

Node.js Module

Module in Node.js is a simple or complex functionality organized in single or multiple JavaScript files which can be reused throughout the Node.js application.

Each module in Node.js has its own context, so it cannot interfere with other modules or pollute global scope. Also, each module can be placed in a separate .js file under a separate folder.

Node.js implements CommonJS modules standard. CommonJS is a group of volunteers who define JavaScript standards for web server, desktop, and console application.

Node.js Module Types

Node.js includes three types of modules:

1. Core Modules
2. Local Modules

3. Third Party Modules

Node.js Core Modules

- Node.js is a light weight framework. The core modules include bare minimum functionalities of Node.js. These core modules are compiled into its binary distribution and load automatically when Node.js process starts. However, you need to import the core module first in order to use it in your application.
- The following table lists some of the important core modules in Node.js.

Core Module	Description
http	http module includes classes, methods and events to create Node.js http server.
url	url module includes methods for URL resolution and parsing.
querystring	querystring module includes methods to deal with query string.
path	path module includes methods to deal with file paths.
fs	fs module includes classes, methods, and events to work with file I/O.
util	util module includes utility functions useful for programmers.

Loading Core Modules

1. HTTP module

- The http module is a core module in Node.js that provides the functionality to create HTTP servers and make HTTP requests. It is one of the fundamental building blocks for building web applications and services in Node.js. This module supports both server-side and client-side operations.
- Key Features
  - Creating HTTP Servers: Easily create HTTP servers that can handle incoming requests and send responses.
  - Making HTTP Requests: Facilitate making HTTP requests to other servers for client-side operations.
- Importing the Module
  - const http = require('http');
- Creating an HTTP Server

- The http.createServer() method in Node.js is a fundamental part of the HTTP module that allows you to create an HTTP server. This server listens for incoming requests and responds to them using a request listener function.
- Parameters: requestListener(optional) - A function that will be automatically added to the 'request' event. This function is invoked each time the server receives an HTTP request. This function accepts two parameters:
  - req (http.IncomingMessage): Represents the incoming request.
  - res (http.ServerResponse): Represents the outgoing response.

- Request (IncomingMessage)

Properties:

req.method	The HTTP method used for the request e.g., 'GET', 'POST'
req.url	The URL of the request.
req.headers	An object containing the request headers.

Methods:

req.on('data',callback):	Adds a listener for the 'data' event, which is emitted when a chunk of data is available.

- Response (http.ServerResponse)

Properties:

res.statusCode	The HTTP status code of the response (default is 200).
res.statusMessage	The HTTP status message associated with the status code (default is 'OK').

Methods:

res.setHeader(name, value)	Sets a single header value for the response
res.write(chunk[, encoding])	Sends a chunk of the response body.
res.end([data[, encoding]][, callback])	Signals that the response is complete and can optionally include the final data to send. - data (string | Buffer, optional): The final data to send. - encoding (string, optional): The encoding of the data (default is 'utf8'). - callback (function, optional): A function called when the response is finished.

- Example:

```
var http = require('http')
http.createServer((req,res)=>{
    // Set the status code and content type
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain');

    if(req.method=='GET' && req.url=='/'){
        res.end("Welcome")
    }else if(req.method=='GET' && req.url=='/MCA'){
        res.end("MCA")
    }else{
        res.statusCode = 404
        res.end("Sorry File not found")
    }

}).listen(1212,()=>{
    console.log('server running on port 1212')
})
```

2. fs module

- The `fs` module in Node.js provides a comprehensive API for interacting with the file system. It offers both asynchronous and synchronous methods for reading, writing, appending, deleting files, and managing directories.

Key Features

- File Operations: Reading, writing, opening, closing, renaming, deleting files.
- Directory Operations: Creating, reading, deleting directories.
- Asynchronous and Synchronous Methods: Each operation can be performed asynchronously or synchronously.

- Importing the Module
  - const fs = require('fs');
- **Methods**

fs.readFile(path, options, callback)	**file**: The path to the file. **path**: The path to the file. **options**: An optional parameter that can include the encoding and flag. **callback**: A callback function that is called with the file data or an error.
fs.writeFile(file, data, options, callback)	
fs.appendFile(file, data, options, callback)	
fs.unlink(path, callback)	
fs.mkdir(path, options, callback)	**options**: An optional parameter that can include `recursive` to create parent directories if needed.
fs.readdir(path, options, callback)	An optional parameter that can include `encoding` and `withFileTypes`.
fs.readFileSync(path, options)	Handle the file system module synchronously.
fs.writeFileSync(file, data, options)	
fs.appendFileSync(file, data, options)	
fs.unlinkSync(path)	
fs.mkdirSync(path, options)	
fs.readdirSync(path, options)	

Examples:

Import fs module

```
const fs = require('fs');
const filePath = 'data.txt';
```

Read from a File

```
fs.readFile(filePath, 'utf8', (err, data) => {
    if (err) throw err;
    console.log('File content:', data);
});
```

Write to file

```
fs.writeFile(filePath, 'Hello, this is a test file!', (err) => {
    if (err) throw err;
    console.log('File written successfully.');
});
```

Append file

```
fs.appendFile(filePath, '\nAppended content.', (err) => {
    if (err) throw err;
    console.log('Content appended successfully.');
});
```

Delete file

```
fs.unlink(filePath, (err) => {
    if (err) throw err;
    console.log('File deleted successfully.');
});
```

3. Url Module

- The `url` module in Node.js provides utilities for URL resolution and parsing. This module allows for parsing URLs, constructing URLs, resolving URLs, and generating URLs.
- Importing the module:
    - const url = require('url');

Methods of url module:

url.parse (urlString, [parseQueryString], [slashesDenoteHost])	**urlString**: The URL string to parse. **parseQueryString** (optional): A boolean indicating whether to parse the query string into an object. Default is `false`. **slashesDenoteHost** (optional): A boolean indicating whether slashes in the path should be treated as the beginning of the path. (host/path), default is `false`.

url.format(urlObject)	The `url.format` method takes a URL object and returns a formatted URL string. **urlObject**: An object representing the URL components.
url.resolve(from, to)	The `url.resolve` method resolves a target URL relative to a base URL. **from**: The base URL. **to**: The target URL to resolve.

Properties of url module:

Property	Description	Example
href	The full URL string that was originally parsed.	`'http://www.example.com:8080/path/name?foo=bar#hash'`
protocol	The protocol scheme of the URL (e.g., 'http:', 'https:').	`'http:'`
slashes	A boolean indicating whether the URL includes a double slash (`//`) after the protocol.	`true`
auth	The authentication information portion of the URL (e.g., 'user').	`null`
host	The full host portion of the URL, including the port number if specified.	`'www.example.com:8080'`
port	The port number specified in the URL.	`'8080'`
hostname	The hostname portion of the URL, without the port.	`'www.example.com'`
hash	The fragment identifier portion of the URL, including the leading `#`.	`'#hash'`
search	The query string portion of the URL, including the leading `?`.	`'?foo=bar'`
query	The query string as a string or an object if `parseQueryString` is set to `true`.	Example (as a string): `'foo=bar'` Example (as an object, if `parseQueryString` is `true`): `{ foo: 'bar' }`
pathname	The path section of the URL.	`'/path/name'`
path	The concatenation of `pathname` and `search`.	`'/path/name?foo=bar'`
href	The full URL that was originally parsed.	`'http://www.example.com:8080/path/name?foo=bar#hash'`

Example:

```
var url = require('url');
var adr = 'http://localhost:8080/default.htm?year=2023&month=february';
var q = url.parse(adr, true);

console.log(q.host); //returns 'localhost:8080'
console.log(q.pathname); //returns '/default.htm'
console.log(q.search); //returns '?year=2023&month=february'

var qdata = q.query; //returns an object: { year: 2023, month: 'february' }
console.log(qdata.month); //returns 'february'
```

Node.js Local Module

- Local modules are modules created locally in your Node.js application. These modules include different functionalities of your application in separate files and folders.
- You can also package it and distribute it via NPM, so that Node.js community can use it. For example, if you need to connect to MongoDB and fetch data then you can create a module for it, which can be reused in your application.

Export Module in Node.js

- The module.exports is a special object which is included in every JavaScript file in the Node.js application by default. The module is a variable that represents the current module, and exports is an object that will be exposed as a module. So, whatever you assign to module.exports will be exposed as a module.

Export Literals and functions

- As mentioned above, exports are an object. So it exposes whatever you assigned to it as a module. For example, if you assign a string literal then it will expose that string literal as a module.
- The following example exposes simple string message as a module in Message.js.

```
module.exports = 'Hello world';
```

- import this message module and use it as shown below.

```
var msg = require('./Message.js');
console.log(msg);
```

Writing Simple Module

- Let's write simple logging module which logs the information, warning or error to the console.
- In Node.js, module should be placed in a separate JavaScript file. So, create a Log.js file and write the following code in it.

```
var log = {
        info: function (info) {
           console.log('Info: ' + info);
        },
        warning:function (warning) {
           console.log('Warning: ' + warning);
        },
        error:function (error) {
           console.log('Error: ' + error);
        }
  };

  module.exports = log
```

- In the above example of logging module, we have created an object with three functions - info(), warning() and error(). At the end, we have assigned this object to **module.exports**. The module.exports in the above example exposes a log object as a module.
- The *module.exports* is a special object which is included in every JS file in the Node.js application by default. Use **module.exports** or **exports** to expose a function, object or variable as a module in Node.js.

Loading Local Module

- To use local modules in your application, you need to load it using require() function in the same way as core module. However, you need to specify the path of JavaScript file of the module.
- The following example demonstrates how to use the above logging module contained in Log.js.

```
var myLogModule = require('./Log.js');
myLogModule.info('Node.js started');
```

- In the above example, app.js is using log module. First, it loads the logging module using require() function and specified path where logging module is stored. Logging module is contained in Log.js file in the root folder. So, we have specified the path './Log.js' in the require() function. The '.' denotes a root folder.
- The require() function returns a log object because logging module exposes an object in Log.js using module.exports. So now you can use logging module as an object and call any of its function using dot notation e.g myLogModule.info() or myLogModule.warning() or myLogModule.error()

NodeJS third party modules:

Third-party modules: Third-party modules are modules that are available online using the Node Package Manager(NPM). These modules can be installed in the project folder or globally. Some of the popular third-party modules are Mongoose, express, angular, and React.

- Examples:
  - **Express framework -** flexible Node js web application framework
  - **Socket.io -** real-time bidirectional event-based communication
  - **Jade -** high-performance template engine
  - **mongoDB -**node.js driver for MongoDB.
  - **Restify -** similar to express for building REST APIs
  - **Bluebird -**promise library

What are the ways to deal with json content in javascript?

- **JavaScript** facilitates the capability to store and transport the data while sending the data from a server to a web page with the help of JSON.
- **JSON (JavaScript Object Notation)** is a language-independent lightweight data interchange format, easy to read and write.
- JSON provides different methods that help to convert the JS object to a string or provides parsing of a JSON string, which is written in a JSON format to a JavaScript object.
- For this, JSON.parse() & JSON.stringify methods are used. In this article, we will see JSON.parse() method & JSON.stringify method, their implementation & difference between them.

JSON.stringify() Method

- It is used to convert JavaScript objects into JSON strings. This method only takes a single argument, and that argument is an object to be converted into JSON strings.
- It works oppositely to **JSON.parse()** method.
- JSON.stringify can take replacer parameters as a function, we can write logic on key-value pairs of objects.
- While receiving data from a web server (like a node server), data has to be in the string format, & after parsing data with JSON.parse(), the data become an Object. The format of the Date is not allowed in JSON, So you can include the Date in the string.
- Syntax: JSON.stringify(data,replacer,space)
  - Data: JSON Object variable or data object
  - Replacer:
  - String

- The first one is a replacer function. The second is a String or Number value to use as a space in the returned string.

Examples (JSON.stringify())

```
const myInfo = {
   Name: "Bob",
   Age:22,
   Department : "Computer Science and Engineering"
 }
 const Obj = JSON.stringify(myInfo);
 console.log(Obj)
```

```
let userObj = {

   name: "Sammy",
   email: "sammy@example.com",
   plan: "Pro"
 };

 function replacer(key, value) {
   console.log(typeof value);
   if (key === 'email') {
     return undefined;
   }
   return value;
 }

 let userStrReplacer = JSON.stringify(userObj, replacer);

 console.log(userStrReplacer);
```

JSON.parse() Method

- It is used to convert a JSON string into a JavaScript Object.
- JavaScript objects are basically storage that store some data of that respective class.
- It takes a single argument, which is the JSON string to be parsed.
- Note that the string to be parsed must be in backticks ‘ ’ and all the words must be in double cots.
- When sending data to a web server, The data has to be in string format. It can be used to store the data in the local storage of the browser because the browser store data in local storage in key-value pairs.

```
const myInfo = `{

   "Name": "GFG",
   "Age":22,
   "Department" : "Computer Science and Engineering"
}`

const Obj = JSON.parse(myInfo);
console.log(Obj.Name)
console.log(Obj.Age)
```

JSON.stringify() Method	*JSON.parse() Method*
Convert data JavaScript object into JSON string.	Convert a JSON string into a JavaScript Object.
The string must be wrapped in double cots. example: "String".	No need to wrap the string in double cots.
while sending data to a web server, data has to be in string format.	While receiving data from the web server data has to be in the string.
The key must be a **string** with double quotes.	Keys can be anything such as String, Number, or Character, but the object must be wrapped in quotes i.e '{key:, value}'.
It allow 2 extra (optional) parameters, i.e value, replacer, and space, for eg., JSON.stringify(value, replacer, space).	It allow one extra (optional) parameter, i.e reviewer, for eg, JSON.parse(value, reviewer).